

PyCRAM Robot API Technical Documentation

This document provides detailed technical specifications for the PyCRAM Robot API endpoints, including parameters, return values, data types, and default behaviors.

API Overview

The PyCRAM Robot API exposes a single HTTP endpoint `/execute`` that accepts POST requests. The request body must contain a JSON object with the following structure:

```
```json
{
 "command": "command_name",
 "params": {
 "param1": value1,
 "param2": value2,
 ...
 }
}
```
```

Where ``command_name`` is one of the available commands documented below, and the ``params`` object contains the parameters specific to that command.

All responses follow this general format:

```
```json
{
 "status": "success|error",
```

```
"message": "Human-readable message",
"payload": {
 // Command-specific response data
}
}
...
```

## ## API Endpoints

### ### 1. spawn\_objects

Creates an object in the simulation environment.

**\*\*Source\*\*:** `robot\_actions\_api.py:183-266`

#### ##### Parameters

Parameter	Type	Required	Default	Description
-----	-----	-----	-----	-----
`object_choice`	string	No	"bowl"	Type of object to spawn ("cereal", "milk", "spoon", "bowl", or custom)
`coordinates`	array[number]	No	[1.4, 1.0, 0.95]	[x, y, z] coordinates for object placement
`color`	string	No	Random from ["red", "blue", "green", "yellow", "black"]	Color for the object
`name`	string	No	`object_choice` + random number (0-100)	Custom name for the object

#### ##### Return Value

**\*\*Success Response\*\*:**

```
```json
{
  "status": "success",
  "message": "Object '{obj_name}' created successfully",
  "object": {
    "name": "string",
    "type": "string",
    "file": "string",
    "pose": [number, number, number],
    "color": "string"
  }
}
```
```

**\*\*Error Response\*\*:**

```
```json
{
  "status": "error",
  "message": "Error message"
}
```
```

#### #### Error Conditions

- If coordinates don't have exactly 3 values
- If file for the specified object type is not found
- Any exception during object creation

#### #### Implementation Notes

- For standard objects (cereal, milk, spoon, bowl), uses predefined ontology classes from PyCRAM
- For custom objects, uses `ObjectType.GENERIC` and assumes STL file with same name exists
- Creates a `ColorWrapper` object to handle color conversion to RGBA values
- Logs created object information to console

### 2. move\_robot

Moves the robot to specified coordinates.

**Source**: `robot\_actions\_api.py:97-124`

#### Parameters

| Parameter     | Type          | Required | Default   | Description                      |
|---------------|---------------|----------|-----------|----------------------------------|
| `coordinates` | array[number] | No       | [0, 0, 0] | [x, y, z] coordinates to move to |

#### Return Value

**Success Response**:

```

```json
{
  "status": "success",
  "message": "Robot moved to coordinates [x, y, z]",
  "coordinates": [number, number, number]
}
```

```

**\*\*Error Response\*\*:**

```
```json
{
  "status": "error",
  "message": "Error message"
}
```
```

#### ##### Implementation Notes

- Uses the `simulated\_robot` context manager for robot control
- Creates a `Pose` object from the coordinates
- Resolves and performs a `NavigateAction` to move the robot
- Logs navigation progress to console

#### ### 3. pickup\_and\_place

Picks up an object and places it at a target location.

**\*\*Source\*\*:** `robot\_actions\_api.py:126-269`

#### ##### Parameters

| Parameter                     | Type          | Required | Default             | Description                                               |
|-------------------------------|---------------|----------|---------------------|-----------------------------------------------------------|
| ----- ----- ----- ----- ----- |               |          |                     |                                                           |
| `object_name`                 | string        | No       | "cereal"            | Name of the object to pick up                             |
| `target_location`             | array[number] | No       | [1.4, 1.0, 0.95]    | [x, y, z] coordinates for placement                       |
| `arm`                         | string        | No       | Automatic selection | Which arm to use ('left', 'right', or null for automatic) |

#### #### Return Value

**\*\*Success Response\*\*:**

```
```json
{
  "status": "success",
  "message": "Successfully picked up {object_name} and placed at {target_location}",
  "object": "string",
  "target_location": [number, number, number],
  "arm_used": "string"
}
```
```

**\*\*Error Response\*\*:**

```
```json
{
  "status": "error",
  "message": "Error message"
}
```
```

#### #### Error Conditions

- If the object is not found in the world
- If no reachable arms are available for the object
- If placement location resolution fails
- Any exception during pickup or place operations

#### #### Implementation Notes

- Uses case-insensitive object name lookup with fallback strategies
- Parks arms and adjusts torso height prior to pickup
- Uses `CostmapLocation` to find reachable positions for the robot
- Has fallback placement logic if the costmap resolution times out
- Implements a timeout mechanism for placement resolution (5 seconds)
- Uses `ParkArmsAction` after placement to reset arm positions

#### ### 4. robot\_perceive

Makes the robot perceive its environment.

**\*\*Source\*\***: `robot\_actions\_api.py:271-337`

##### ##### Parameters

| Parameter                     | Type   | Required | Default | Description                              |
|-------------------------------|--------|----------|---------|------------------------------------------|
| ----- ----- ----- ----- ----- |        |          |         |                                          |
| `perception_area`             | string | No       | "table" | Area to perceive ('table', 'room', etc.) |

##### ##### Return Value

**\*\*Success Response\*\***:

```
```json
{
  "status": "success",
  "message": "Robot perceived {n} objects in {perception_area}",
  "perceived_objects": [
    {
```

```

    "name": "string",
    "type": "string",
    "pose": "string"
  },
  ...
]
}
```

```

```

Error Response:
```json
{
  "status": "error",
  "message": "Error message"
}
```

```

#### #### Implementation Notes

- Uses `LookAtAction` to orient the robot toward objects if possible
- For table/counter/surface perception, attempts to detect common object types
- Has fallback mechanism to directly access world objects if perception fails
- Filters out the robot itself and environment objects from perception results

#### ### 5. transport\_object

Transports an object to a target location.

**Source**: `robot\_actions\_api.py:340-376`



#### #### Parameters

| Parameter                     | Type          | Required | Default         | Description                               |
|-------------------------------|---------------|----------|-----------------|-------------------------------------------|
| ----- ----- ----- ----- ----- |               |          |                 |                                           |
| `object_name`                 | string        | No       | "cereal"        | Name of the object to transport           |
| `target_location`             | array[number] | No       | [1.0, 1.0, 0.8] | [x, y, z] coordinates for target location |
| `arm`                         | string        | No       | "right"         | Which arm to use ('left', 'right')        |

#### #### Return Value

**\*\*Success Response\*\*:**

```
```json
{
  "status": "success",
  "message": "Successfully transported {object_name} to {target_location}",
  "object": "string",
  "target_location": [number, number, number],
  "arm_used": "string"
}
```
```

**\*\*Error Response\*\*:**

```
```json
{
  "status": "error",
  "message": "Error message"
}
```

...

Implementation Notes

- Converts arm string to the appropriate `Arms` enum value
- Creates a `BelieveObject` designator to reference the object
- Creates a `Pose` from the target location coordinates
- Uses `TransportAction` to perform the transport operation
- Logs transport progress to console

6. get_camera_images

Captures images from the robot's camera and returns them as base64-encoded strings.

****Source**:** `robot\_actions\_api.py:636-693`

Parameters

Parameter	Type	Required	Default	Description
----- ----- ----- ----- -----				
`target_distance`	number	No	2.0	Distance in meters to the target point

Return Value

****Success Response**:**

```
```json
{
 "status": "success",
 "message": "Camera images captured successfully",

```

```

"images": {
 "color_image": "string (base64)",
 "depth_image": "string (base64)",
 "segmentation_mask": "string (base64)"
}
}
```

```

****Error Response**:**

```

```json
{
 "status": "error",
 "message": "Error message"
}
```

```

Implementation Notes

- Converts float target distance to the appropriate value
- Uses `capture\_camera\_image` to get RGB, depth, and segmentation images
- Encodes the images as base64 strings:
 - For RGB images: converts to uint8 format, encodes as PNG, then to base64
 - For depth images: normalizes to 0-255 range, encodes as PNG, then to base64
- Uses OpenCV for image encoding

7. get_enhanced_camera_images

Captures enhanced visualization of images from the robot's camera.

****Source**:** `robot\_actions\_api.py:695-734`

Parameters

| Parameter | Type | Required | Default | Description |
|-------------------|--------|----------|---------|--|
| `target_distance` | number | No | 2.0 | Distance in meters to the target point |

Return Value

****Success Response**:**

```
```json
{
 "status": "success",
 "message": "Enhanced camera images captured successfully",
 "images": {
 "color_image": "string (base64)",
 "depth_image": "string (base64)",
 "segmentation_mask": "string (base64)"
 }
}
```
```

****Error Response**:**

```
```json
{
 "status": "error",
 "message": "Error message"
}
```

```
}
'''
```

#### ##### Implementation Notes

- Similar to `get\_camera\_images` but uses matplotlib to create enhanced visualizations
- Depth images are rendered with the 'viridis' colormap for better visualization
- Images are saved to BytesIO buffers and then encoded to base64
- Returns same data structure but with enhanced image visualizations

#### ### 8. calculate\_object\_distances

Calculates distances between objects in the world.

**\*\*Source\*\*:** `robot\_actions\_api.py:81-95`

#### ##### Parameters

Parameter	Type	Required	Default	Description
----- ----- ----- ----- -----				
`source_object`	string	No	null	Name of source object (if null, calculates distances between all pairs)
`target_objects`	array[string]	No	null	List of target object names (if null, uses all objects)
`exclude_types`	array[string]	No	["floor", "wall", "ceiling", "ground"]	List of object types to exclude

#### ##### Return Value

When `source\_object` is specified:

```
```json
```

```
{
  "object_name1": distance1,
  "object_name2": distance2,
  ...
}
```

...

When calculating all pairwise distances:

```
```json
{
 "object1-to-object2": distance,
 "object1-to-object3": distance,
 ...
}
```

...

#### #### Implementation Notes

- Filters objects based on exclusion list
- Can calculate distances from one source object to all others
- Can calculate all pairwise distances between objects
- Uses Euclidean distance formula for 3D positions
- Returns empty dictionary if world is not initialized

#### ### 9. look\_at\_object

Makes the robot look at the specified object.

**\*\*Source\*\*:** `robot\_actions\_api.py:378-408`

#### #### Parameters

Parameter	Type	Required	Default	Description
----- ----- ----- ----- -----				
`obj_name`	string	Yes	N/A	Name of the object to look at

#### #### Return Value

**\*\*Success Response\*\*:**

```
```json
{
  "status": "success",
  "message": "Robot is now looking at '{obj_name}'"
}
```
```

**\*\*Error Response\*\*:**

```
```json
{
  "status": "error",
  "message": "Object '{obj_name}' is not in the environment"
}
```
```

or

```
```json
{
  "status": "error",
```

```
"message": "Robot vision failed to reach '{obj_name}'"
}
...
```

Error Conditions

- If the object is not found in the environment
- If the robot cannot successfully look at the object (LookAtGoalNotReached)

Implementation Notes

- Converts object name to lowercase for case-insensitive comparison
- Uses `BelieveObject` designator to resolve the object
- Uses `LookAtAction` to make the robot look at the object
- Handles `LookAtGoalNotReached` exception explicitly

10. detect_object

Detects an object in the robot's environment and returns information about it.

****Source**:** `robot\_actions\_api.py:12-80`

Parameters

Parameter	Type	Required	Default	Description
-----	-----	-----	-----	-----
`object_name`	string	No	null	Name of the object to detect (if null, detects any visible object)
`location`	object or array	No	null	Location to search for objects (can be a Pose object or position array)

Return Value

****Success Response (single object)**:**

```
```json
{
 "status": "success",
 "message": "Successfully detected object: {object_name}",
 "object": {
 "name": "string",
 "type": "string",
 "position": {
 "x": number,
 "y": number,
 "z": number
 },
 "color": "string"
 }
}
```
```

****Success Response (multiple objects)**:**

```
```json
{
 "status": "success",
 "message": "Successfully detected {n} object(s)",
 "objects": [
 {
 "name": "string",

```

```
"type": "string",
 "position": {
 "x": number,
 "y": number,
 "z": number
 },
 "color": "string"
},
...
]
}
...
```

**\*\*Error Response\*\*:**

```
```json  
{  
  "status": "error",  
  "message": "Error message"  
}  
...
```

Error Conditions

- If no objects matching the criteria are detected
- If the world is not initialized
- Any exception during detection

Implementation Notes

- First tries to orient the robot camera toward a central object

- If `object\_name` is provided, searches for that object directly in the world
- Else uses `DetectAction` with `DetectionTechnique.ALL` to find objects
- Handles multiple detection results appropriately
- Extracts detailed information about detected objects including position and color

11. move_and_rotate

Moves the robot to a specified location and/or rotates it to a specified orientation.

****Source**:** `robot\_actions\_api.py:736-795`

Parameters

Parameter	Type	Required	Default	Description
----- ----- ----- -----				
`location`	array[number]	No	Current position	[x, y, z] coordinates to move to
`angle`	number	No	Current angle	Angle in degrees to rotate around Z axis

Return Value

****Success Response**:**

```
```json
{
 "status": "success",
 "message": "Robot moved to coordinates {location} with orientation {orientation}",
 "position": [number, number, number],
 "orientation": [number, number, number, number]
}
```

```
'''
```

```
Error Response:
```

```
```json
```

```
{
```

```
    "status": "error",
```

```
    "message": "Error message"
```

```
}
```

```
'''
```

Error Conditions

- If location is not a list of 3 values
- If world is not initialized
- If robot is not found in world
- Any exception during movement

Implementation Notes

- Gets current robot pose if location or angle is not provided
- Uses `euler\_to\_quaternion` to convert angle to quaternion orientation
- Creates a `Pose` object with position and orientation
- Uses `NavigateAction` to move and rotate the robot
- Logs navigation progress to console

12. move_torso

Moves the robot's torso to a specified position.

```
**Source**: `robot_actions_api.py:797-826`
```

Parameters

Parameter	Type	Required	Default	Description
`position`	string	No	"high"	Position to move the torso to ("low", "high")

Return Value

****Success Response**:**

```
```json
{
 "status": "success",
 "message": "Torso moved to {position} position",
 "position": "string"
}
```
```

****Error Response**:**

```
```json
{
 "status": "error",
 "message": "Invalid torso position: {position}. Valid options are: low, high."
}
```
```

Error Conditions

- If an invalid torso position is provided

- Any exception during torso movement

Implementation Notes

- Maps string position to the appropriate `TorsoState` enum
- Valid positions are "low" and "high" (case-insensitive)
- Uses `MoveTorsoAction` to move the robot's torso
- Logs torso movement to console

13. park_arms

Moves the robot's arm(s) to the pre-defined parking position.

****Source**:** `robot\_actions\_api.py:828-862`

Parameters

| Parameter | Type | Required | Default | Description |
|-------------------------------|--------|----------|---------|---------------------------------------|
| ----- ----- ----- ----- ----- | | | | |
| `arm` | string | No | "both" | Arm to park ("left", "right", "both") |

Return Value

****Success Response**:**

```
```json
{
 "status": "success",
 "message": "Successfully parked arm(s): {arm}",
 "arm": "string"
}
```

```
}
'''
```

**\*\*Error Response\*\*:**

```
```json  
{  
  "status": "error",  
  "message": "Invalid arm: {arm}. Valid options are: left, right, both."  
}  
'''
```

Error Conditions

- If an invalid arm value is provided
- Any exception during arm movement

Implementation Notes

- Maps string arm value to the appropriate `Arms` enum values
- Valid arm values are "left", "right", "both" (case-insensitive)
- If no arm is specified, parks both arms
- Uses `ParkArmsAction` to move the robot's arms to parking position
- Logs arm movement to console

API Server Implementation

The API server (`api.py`) uses FastAPI to expose the `/execute` endpoint that routes requests to the appropriate function in `robot\_actions\_api.py`. It also initializes the environment at startup.

Key Components

Environment Initialization

****Source**:** `api.py:28-69`

- Initializes the simulation environment before starting the server
- Imports environment functions dynamically to prevent auto-execution
- Creates environment based on user selection (kitchen, apartment)
- Spawns default objects if in kitchen environment:
 - Blue spoon at [1.4, 1, 0.95]
 - Red cereal at [1.4, 0.8, 0.95]
 - Green milk at [1.4, 0.6, 0.95]

Execute Command Endpoint

****Source**:** `api.py:100-157`

- Path: `/execute`
- Method: POST
- Accepts: JSON object with `command` and `params`
- Routes to the appropriate function based on the command
- Catches and returns exceptions as error responses

Testing Implementation

The test module (`test\_api.py`) provides functions to test each API endpoint. Each test function constructs an appropriate payload, sends it to the API server, and prints the response.

Test Functions

test_camera

****Source**:** `test\_api.py:15-56`

Tests the `get\_camera\_images` endpoint and saves the resulting images to disk.

test_move_robot

****Source**:** `test\_api.py:59-65`

Tests the `move\_robot` endpoint with coordinates [0, 0, 0].

test_pickup_and_place

****Source**:** `test\_api.py:68-74`

Tests the `pickup\_and\_place` endpoint with a cereal object and target location [1.4, 0.2, 0.95].

test_robot_perceive

****Source**:** `test\_api.py:77-83`

Tests the `robot\_perceive` endpoint with perception area "sink".

test_transport_object

****Source**:** `test\_api.py:86-92`

Tests the `transport\_object` endpoint with a cereal object and target location [1.4, 0.2, 0.95].

test_calculate_object_distances

****Source**:** `test\_api.py:95-101`

Tests the `calculate\_object\_distances` endpoint with source object "cereal".

```
##### test_look_at_object
```

```
**Source**:`test_api.py:104-110`
```

Tests the `look\_at\_object` endpoint with object name "cereal".

```
##### test_detect_object
```

```
**Source**:`test_api.py:113-119`
```

Tests the `detect\_object` endpoint with no parameters.

```
##### test_move_and_rotate
```

```
**Source**:`test_api.py:122-128`
```

Tests the `move\_and\_rotate` endpoint with location [0, 0, 0] and angle 60 degrees.

```
##### test_move_torso
```

```
**Source**:`test_api.py:131-137`
```

Tests the `move\_torso` endpoint with position "high".

```
##### test_park_arms
```

```
**Source**:`test_api.py:141-147`
```

Tests the `park\_arms` endpoint with no parameters (both arms).

```
##### test_enhanced_camera
```

****Source**:** `test\_api.py:150-182`

Tests the `get\_enhanced\_camera\_images` endpoint and saves the resulting images to disk.

test_spawn_objects

****Source**:** `test\_api.py:185-195`

Tests the `spawn\_objects` endpoint with object "cereal" at coordinates [1.4, 0.4, 0.95].

Common Response Fields

All API responses include the following common fields:

- `status`: String value "success" or "error" indicating the result of the operation
- `message`: Human-readable message explaining the result
- Additional fields specific to each endpoint